3DG

CUSTOM ICON REPLACEMENT GUIDE

A project-specific workflow for replacing the Windows Segoe MDL2 glyphs, asset and panel symbols, executable icon, runtime window icon, and launcher brand mark with artwork you own.

Built for this repository Paths and code examples match D:/C++_Projects/3DGEngine and the current ImGui 1.92+ font-atlas setup.

Outcome	What you will replace
Editor controls	Toolbar, menus, buttons, tabs, panels, asset types, and groups
Application branding	Editor and launcher executable icon, taskbar/window icon, launcher mark
Build safety	Correct font merge sizing, asset copying, fallback behavior, and validation

Version 1.0 | Generated from the engine source on 2026-08-09

1. Understand the current icon pipeline

The engine does not use one icon source. It currently has three independent systems. Replacing only the .ico file will not change toolbar or Content Browser icons.

System	Current source	Main files	Replacement format
ImGui semantic icons	Windows Segoe MDL2 font merged into the default ImGui font	engine/src/ui/ImGuiLayer.cpp; editor/include/EditorIcons.h	Custom TTF/OTF icon font
Panel and asset mappings	Semantic constants such as Play, World, Material, Folder	EditorPanelIcons.h; EditorAssetIcons.h	Reuse custom font glyph constants
Executable icon	Windows resource referencing 3DGEditor.ico	editor/assets/branding/3DGEditor.rc	Multi-resolution ICO
Runtime window icon	Procedurally generated RGBA cube mark	EditorBranding.cpp; EditorBranding.h	Embedded RGBA or decoded PNG
Launcher header mark	Lines and a circle drawn with ImDrawList	editor/src/launcher/LauncherApp.cpp	Custom vector drawing or texture

The important source locations are:

```
editor/include/EditorIcons.h
editor/include/EditorAssetIcons.h
editor/include/EditorPanelIcons.h
engine/src/ui/ImGuiLayer.cpp
editor/assets/branding/3DGEditor.ico
editor/assets/branding/3DGEditor.rc
editor/src/app/EditorBranding.cpp
editor/src/launcher/LauncherApp.cpp
```

Recommended migration order Replace the ImGui icon font first, verify controls, then replace the executable and runtime branding. This keeps failures isolated.

2. Design and export your icon family

Create icons as monochrome vector shapes. ImGui applies color separately, so the glyph itself should normally contain a single filled silhouette. Use consistent proportions before generating the font.

Design rule	Recommended value	Reason
Artboard	1024 x 1024 or 1000 x 1000	Easy scaling into a font em square
Safe area	Keep artwork inside the central 80%	Avoid clipped strokes and cramped buttons
Stroke treatment	Convert strokes to filled outlines	Font exporters handle fills more reliably
Visual weight	One consistent stroke/shape weight	Mixed weights look uneven at 13 px
Small-size test	Preview at 13, 16, 20, and 24 px	Editor icons are primarily small
Naming	lowercase semantic names	Keeps source mapping readable

Minimum semantic set

Create at least: add, archive, back, code, character, copy, cut, delete, document, download, edit, folder, image, layers, leaf, link, music, open, palette, paste, play, refresh, screen, settings, star, stop, save, up, video, and world.

Do not use emoji Emoji may select a color font, vary by operating system, and fail in the ImGui atlas. Use your own monochrome outline/fill glyphs.

3. Build a custom icon font

An icon font is the least disruptive replacement because the editor already renders icons as text. You can use FontForge, IcoMoon, Fontello, or another font generator. Export a TrueType font named 3DGEditorIcons.ttf.

Assign Private Use Area codepoints

Name	Codepoint	C++ literal
add	U+E000	u8"\uE000"
archive	U+E001	u8"\uE001"
back	U+E002	u8"\uE002"
code	U+E003	u8"\uE003"
character	U+E004	u8"\uE004"
copy	U+E005	u8"\uE005"
cut	U+E006	u8"\uE006"
delete	U+E007	u8"\uE007"
document	U+E008	u8"\uE008"
download	U+E009	u8"\uE009"
edit	U+E00A	u8"\uE00A"
folder	U+E00B	u8"\uE00B"
image	U+E00C	u8"\uE00C"
layers	U+E00D	u8"\uE00D"
leaf	U+E00E	u8"\uE00E"
link	U+E00F	u8"\uE00F"
music	U+E010	u8"\uE010"
open	U+E011	u8"\uE011"
palette	U+E012	u8"\uE012"
paste	U+E013	u8"\uE013"
play	U+E014	u8"\uE014"
refresh	U+E015	u8"\uE015"
screen	U+E016	u8"\uE016"
settings	U+E017	u8"\uE017"
star	U+E018	u8"\uE018"
stop	U+E019	u8"\uE019"
save	U+E01A	u8"\uE01A"
up	U+E01B	u8"\uE01B"
video	U+E01C	u8"\uE01C"
world	U+E01D	u8"\uE01D"

4. Add the font to the engine

Create this folder and copy your generated font into it:

```
editor/assets/icons/3DGEditorIcons.ttf
```

The existing 3DGEditor post-build command copies the complete editor/assets directory beside the executable. The runtime path will therefore be build/bin/editor/Debug/assets/icons/3DGEditorIcons.ttf.

Replace the Segoe MDL2 path in ImGuiLayer.cpp

Include engine/core/Paths.h, remove the Windows Fonts lookup, and load the project-owned font from the executable directory. Keep the base text font and merged icon font at the same explicit reference size.

```
#include "engine/core/Paths.h"

constexpr float editorFontSize = 13.0f;
ImFontConfig textConfig;
textConfig.SizePixels = editorFontSize;
io.Fonts->AddFontDefault(&textConfig);

const std::filesystem::path iconFontPath =
    std::filesystem::path(engine::ExecutableDir()) /
    "assets" / "icons" / "3DGEditorIcons.ttf";

std::error_code fontError;
if (std::filesystem::is_regular_file(iconFontPath, fontError)) {
    ImFontConfig iconConfig;
    iconConfig.MergeMode = true;
    iconConfig.PixelSnapH = true;
    iconConfig.GlyphMinAdvanceX = editorFontSize;
    iconConfig.OversampleH = 2;
    static const ImWchar iconRanges[] = { 0xE000, 0xE01D, 0 };
    io.Fonts->AddFontFromFileTTF(iconFontPath.string().c_str(),
        editorFontSize, &iconConfig, iconRanges);
}
```

Critical ImGui 1.92+ rule	Do not merge an explicitly sized icon font into a default font that used an implicit reference size. Set textConfig.SizePixels and use that same value for AddFontFromFileTTF.

Cross-platform note

The current Segoe loading block is Windows-only. A project-owned TTF can be loaded on Windows, Linux, and macOS, so place the new loading code outside the _WIN32 guard. Only the executable .ico resource remains Windows-specific.

5. Map your glyphs to editor semantics

Update editor/include/EditorIcons.h. The rest of the UI mostly references semantic names, so changing this one table updates toolbar buttons, context menus, panels, and assets without rewriting every panel.

```
inline constexpr ImWchar kProbeGlyph = 0xE00B; // Folder
inline constexpr const char* Add      = u8"\uE000";
inline constexpr const char* Archive  = u8"\uE001";
inline constexpr const char* Back     = u8"\uE002";
inline constexpr const char* Code     = u8"\uE003";
inline constexpr const char* Contact  = u8"\uE004";
// Continue through U+E01D using your exported mapping.
inline constexpr const char* World    = u8"\uE01D";
```

The probe glyph

Available() asks the current ImGui font whether kProbeGlyph exists. Choose a glyph that is always present in your font, normally Folder. If the probe is wrong, every icon helper intentionally falls back to text labels.

Panel and asset mappings

EditorPanelIcons.h maps panels and groups to semantic constants. EditorAssetIcons.h maps asset types and colors. You only need to edit these files when you want a different semantic symbol, not when merely redrawing the same symbol.

File	Change it when
EditorIcons.h	Codepoints or semantic icon names change
EditorPanelIcons.h	A panel/group should use a different semantic icon
EditorAssetIcons.h	An asset type needs a different icon or color

Preserve ImGui IDs Do not remove the ### or ## portions in WindowLabel and ControlLabel. They keep widget identities stable while the visible icon changes.

6. Replace the executable icon

Create a Windows ICO containing multiple sizes, then replace editor/assets/branding/3DGEditor.ico without changing its filename. The same resource is compiled into both 3DGEditor.exe and 3DGLauncher.exe.

Required sizes	Recommended color format	Purpose
16 x 16	32-bit RGBA	Small lists and title bars
24 x 24	32-bit RGBA	Scaled UI
32 x 32	32-bit RGBA	Taskbar and shortcuts
48 x 48	32-bit RGBA	Explorer views
64 x 64	32-bit RGBA	High-DPI transition size
128 x 128	32-bit RGBA	Large icons
256 x 256	PNG-compressed inside ICO	Extra-large Explorer icon

The resource file should remain:

```
#include <windows.h>
IDI_3DG_EDITOR ICON "3DGEditor.ico"
```

If you rename the ICO, update 3DGEditor.rc. CMake already adds this resource to both Windows targets at editor/CMakeLists.txt lines 89 and 127.

Force Windows to show the new icon

Rebuild both executables. If Explorer still displays the previous artwork, change the executable name temporarily or rebuild the Windows icon cache. This is usually cache behavior, not a failed resource compile.

7. Replace the runtime window and launcher mark

The taskbar/window icon is not read from the ICO at runtime. EditorApp and LauncherApp call editor::branding::Icon(), which currently generates a 64 x 64 cube mark in EditorBranding.cpp.

Option A - keep it embedded

Convert a 64 x 64 transparent PNG into a C++ RGBA byte array and return that data from Icon(). This preserves the current no-file-failure behavior. Keep width, height, and exactly width * height * 4 bytes in RGBA order.

```
WindowIcon BuildIcon() {
    WindowIcon icon;
    icon.width = 64;
    icon.height = 64;
    icon.rgba.assign(kMyIconRgba,
        kMyIconRgba + (64 * 64 * 4));
    return icon;
}
```

Option B - load a PNG from editor assets

Add editor/assets/branding/3DGEditorWindow.png, decode it once inside Icon(), and keep the procedural icon as a fallback. Use the engine image decoder rather than introducing a second decoding library. The returned WindowIcon must own its RGBA bytes for the lifetime of the application.

Launcher header mark

DrawBrandMark() in LauncherApp.cpp is a separate ImDrawList vector drawing. To keep a vector mark, replace its point/line commands with your design. For a full-color logo, upload a texture once during LauncherApp::OnInit(), draw it with ImGui::Image(), and release it during OnShutdown().

Avoid duplicated artwork drift	Use one master SVG. Export the icon font, ICO, 64 px window PNG, and launcher logo from that source so proportions remain consistent.

8. Build and verify

Build commands

```
cmake --build build --config Debug --target 3DGLauncher
cmake --build build --config Debug --target 3DGEditor
```

Building 3DGLauncher also builds its editor dependency. The expected outputs are:

```
build/bin/editor/Debug/3DGLauncher.exe
build/bin/editor/Debug/3DGEditor.exe
build/bin/editor/Debug/assets/icons/3DGEditorIcons.ttf
```

Visual verification checklist

[ ] Launcher opens without a font-atlas or cursor-layout assertion.

[ ] Launcher header, Browse, Create, and Refresh controls show the intended icons.

[ ] Editor main menu, tabs, toolbar, and panel titles show icons.

[ ] Content Browser folders and every asset type have readable symbols.

[ ] Icons remain centered at 100%, 125%, 150%, and 200% Windows scaling.

[ ] Missing-font fallback still leaves readable text labels.

[ ] Editor and launcher executable/taskbar icons use the new artwork.

[ ] Debug and Release builds both copy the custom font beside the executable.

Automated verification

```
cmake --build build --config Debug
ctest --test-dir build -C Debug --output-on-failure
```

9. Troubleshooting

Symptom	Likely cause	Fix
Text appears but icons do not	Font file missing, wrong path, or bad probe glyph	Check copied TTF, iconRanges, and kProbeGlyph
Square boxes	Codepoint is absent from font or excluded from ranges	Inspect font mapping and expand the ImWchar range
MergeMode assertion	Explicit icon size merged into implicit base size	Set textConfig.SizePixels and reuse the exact size
Icons are blurry	Artwork has thin details or poor hinting	Simplify paths, use filled shapes, test PixelSnapH on/off
Buttons collide or duplicate IDs	Visible icon text was used as the only ID	Keep ##/### IDs and PushID scopes
Launcher has icons, editor does not	Different executable asset folder or stale build	Confirm assets/icons exists beside both executables
ICO changed but Explorer did not	Windows icon cache	Clean/rebuild, rename executable temporarily, refresh cache
Linux/macOS has no icons	Font load remained inside _WIN32	Move project-owned TTF loading outside the Windows guard
Runtime window still shows old mark	EditorBranding.cpp still generates cube RGBA	Replace the embedded runtime icon separately from the ICO

Safe fallback policy Keep EditorIcons::Available() and text-label fallbacks. A missing cosmetic font should never make the editor unusable.

10. Migration and rollback checklist

Before changing code

[ ] Save the master SVGs and exported TTF in source control.

[ ] Record every glyph name and codepoint in a mapping file.

[ ] Back up EditorIcons.h, ImGuiLayer.cpp, the ICO, and EditorBranding.cpp.

[ ] Confirm your artwork license permits redistribution with the engine.

During migration

[ ] Add the TTF before changing C++ literals.

[ ] Replace the font path and glyph range.

[ ] Update EditorIcons.h in one atomic change.

[ ] Verify panel and asset mappings.

[ ] Replace ICO, runtime icon, and launcher mark separately.

Rollback

If the editor becomes unreadable, restore EditorIcons.h and the Segoe MDL2 loading block first. The text-label fallback should then recover controls. Restore branding files independently; branding cannot repair a font-atlas failure.

```
git diff -- engine/src/ui/ImGuiLayer.cpp
git diff -- editor/include/EditorIcons.h
git diff -- editor/src/app/EditorBranding.cpp
git diff -- editor/assets/branding/3DGEditor.rc
```

Definition of done	Your icon family is complete when one owned master design produces the custom TTF, executable ICO, runtime window icon, and launcher mark; both Debug and Release builds work without system icon fonts.